

Phase 7: Testing & Quality Assurance

Project Title: AI-Based In-Cabin Monitoring System

1. Test Plan & Strategy

The testing strategy validates both the individual AI modules (Unit Testing) and the combined Flask dashboard pipeline (Integration Testing) under real-world, edge-computing constraints.

- Unit Testing (AI Models): Each model is evaluated on a held-out test split using standard computer vision metrics (mAP, Precision, Recall, AUC).
- Integration Testing: Ensures that background threads (YOLO + MoViNet + MediaPipe) successfully communicate with the Flask web server without race conditions, and that alerts trigger within the 2-second KPI.
- Performance Testing: Benchmarking the system on a target NVIDIA GTX 1650 / 1660 Super to confirm a stable ~30 FPS and sub-50ms inference latency.

2. Accuracy & Performance Metrics

AI Model Evaluation Results

Module	Model Architecture	Core Metric	Score	Notes
Violence Detection	MoViNet A0 + MIL	Accuracy	82.8%	AUC-ROC: 0.7554. Trained on SCVD dataset.
Weapon Detection	YOLOv8n	mAP@50	~0.75	Optimized for real-time edge processing.
Drowsiness Detection	YOLO11n	mAP@50	98.4%	Recall: 95.0%. Evaluated on 498 held-out test images.
Distraction Detection	YOLOv8n	mAP@50	72.2%	Highest precision on Bottles (94%) and Phones (88%).
Face Authentication	InsightFace (ArcFace)	AUC	~0.99	Excellent separability between genuine and impostor pairs.

System Performance

- End-to-End Latency: 100-150ms total (33ms capture + 30ms detection + 10ms encoding + 20ms network + rendering).
- VRAM Usage: ~2.1 GB stable (safely under the 4GB/6GB limits of the GTX 1650/1660).
- Dashboard UI: Successfully polls JSON metrics at 2Hz (every 500ms) while streaming MJPEG smoothly.

3. Bug Reports & Resolutions

Issue Detected	Root Cause	Resolution
MoViNet Dimension Crash	Feeding 224x224 frames into a model trained on 172x172.	Enforced strict 172x172 resizing in the preprocessing pipeline.
Distraction Model Failing on Live Feed	Initial model used EfficientNetB3 for classification, which failed to generalize to live webcam angles.	Pivoted architecture to YOLOv8n for object detection, trained on a newly curated 5.5K image dataset.
Causal Buffer "Remembering" Old Clips	MoViNet causal state buffers were not cleared between separate prediction windows.	Added <code>clean_activation_buffers()</code> call before every forward pass.
MJPEG String Corruption	Line breaks inside byte string literals caused <code>SyntaxError</code> in the Flask generator.	Consolidated byte string concatenation to a single line using raw strings.
PyTorch 2.2.1 Import Error	NumPy 2.0+ changed the C-API, breaking older PyTorch versions.	Pinned the dependency explicitly to <code>numpy==1.24.4</code> in <code>requirements.txt</code> .

Phase 8: Deployment & Execution

1. Deployment Strategy

The system is deployed locally via Edge Computing. It does not rely on cloud infrastructure, ensuring zero latency from network transmission and absolute privacy for the vehicle's occupants.

- Environment: Python Virtual Environment (venv or conda).
- Hardware: Deployed directly to an in-car processing unit equipped with a CUDA-enabled NVIDIA GPU.
- Database: SQLite is used as a lightweight, serverless local database for logging all dashboard events.

2. Installation & Execution Guide

System Requirements

- OS: Ubuntu 22.04 or Windows 10/11
- GPU: NVIDIA GTX 1650 or higher (Minimum 4GB VRAM)
- Dependencies: CUDA Toolkit 11.8 / 12.1, Python 3.11.5

Installation Steps

1. Clone the Repository:

```
git clone https://github.com/Moselhy22/Depi_Ai_Final_Project.git
cd Depi_Ai_Final_Project
```

2. Environment Setup:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. Model Placement:

Ensure all .pt weight files and ArcFace embeddings are placed in their respective models/ directories.

Execution Guide

1. Initialize the Flask Dashboard:

```
python backend/app.py --host 0.0.0.0 --port 5000
```

2. Access the Interface:

Open a web browser and navigate to <http://localhost:5000>.

3. Termination:

Press CTRL+C in the terminal to cleanly terminate background threads, close the SQLite database connection, and shut down the Flask server.